МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

Методичні вказівки до лабораторної роботи № 2

з дисципліни «Вебтехнології та розробка вебзастосувань»

на тему: «Основи JavaScript»
для студентів першого (бакалаврського) рівня вищої освіти спеціальності F3 Комп'ютерні науки освітньо-професійної програми "Комп'ютерні науки (Обчислювальний інтелект смарт-систем)"

Затверджено
на засіданні кафедри
Автоматизованих Систем Управління.
Протокол № 1-2025/2026 « 2 » вересня 2025р.

Львів - 2025

Методичні вказівки до виконання лабораторної роботи №2 «Основи JavaScript» з дисципліни «Вебтехнології та розробка вебзастосувань» для студентів першого (бакалаврського) рівня вищої освіти спеціальності F3 Комп'ютерні науки освітньо-професійної програми "Комп'ютерні науки (Обчислювальний інтелект смарт-систем)" / Укл. Артем Казарян – Львів: Видавництво Національного університету “Львівська політехніка”, 2025. – 20с.

Укладач: асистент кафедри АСУ, д-р філософії Казарян А.Г.

Відповідальний за випуск: Шпак З.Я., к.т.н., доцент

Рецензенти: Березький О.М., д.т.н., професор

Гавриш В.І., д.т.н., професор

Казимира І.Я., к.т.н., доцент

Лабораторна робота №2

Тема: Основи JavaScript

***Мета роботи*:** Навчитись застосовувати основні функціональні можливості мови програмування JavaScript, такі як маніпуляції з елементами вебсторінки, обробка подій, цикли та умови, для реалізації інтерактивності та динамічності вебсторінкам.

Теоретичні відомості

Вступ до JavaScript та його роль у веброзробці

JavaScript — це основна мова програмування, яка живить інтернет з моменту свого виникнення. Почавши свій шлях як легкий сценарій для взаємодії з елементами вебсторінок, сьогодні JavaScript перетворився на універсальну, потужну платформу мов програмування, що охоплює як фронтенд, так і бекенд, завдяки середовищу виконання на сервері. Мова відзначається широким синтаксисом, прототипною моделлю наслідування, підтримкою функціональних і об’єктно-орієнтованих підходів. Вона має гнучкий режим роботи з типами даних, підтримує асинхронні конструкції, події та реактивні підходи. Завдяки цьому JavaScript стала суспільно прийнятою мовою для побудови сучасних інтерактивних вебзастосунків і ресурсів.

Історично JavaScript було створено у 1995 році для додання інтерактивності вебсторінкам — можливості живого реагування браузера на дії користувача, зміни вмісту без перезавантаження. Згодом мова перетворилася на універсальний інструмент розробки. Різноманітні модулі, фреймворки й середовища виконання, зокрема Node.js, зробили JavaScript центральною технологією в екосистемі веброзробки. Сьогодні не існує суттєвих обмежень її застосування: браузерна частина, серверна логіка, десктопні і мобільні додатки, автоматизовані сценарії, інструментальні інтерфейси та багато іншого створюються під її керівництвом.

JavaScript є мовою високого рівня. Це означає, що розробник не працює з дрібними деталями управління пам’яттю чи режимами ядра операційної системи, а може безпосередньо формувати логічні взаємодії, реакцію на події та асинхронне споживання даних. Спочатку браузерне оточення надавало ключові об’єкти: document, window, fetch, XMLHttpRequest, event loops і ін. Ці можливості формували концепцію реактивного інтерфейсу: кожна дія користувача — це подія, у відповідь на яку виконується певний код. Події кліку, навігації, надсилання форми, дії миші і клавіатури — всі вони об'єднані в єдину систему, яка дозволяє побудувати плавну взаємодію.

У сучасних браузерах JavaScript виконується через рушії вдосконалення — найпопулярніші з них це V8 (у Chrome і Node.js), SpiderMonkey (у Firefox),

Chakra (старі версії Edge) і JavaScriptCore (у Safari). Вони компілюють інтерпретований JavaScript в машинний код на льоту, оптимізують сценарії, прискорюють цикл виконання. Наявність таких рушіїв відкриває можливість створювати дійсно продуктивні вебдодатки.

Ключовою галузжю застосування JavaScript є управління об'єктною моделю документа — DOM. Це структура, яка представляє HTML-документ у вигляді ієрархічного дерева. Сценарії JavaScript можуть використовувати DOM API, щоб динамічно створювати, змінювати або видаляти елементи на сторінці, змінювати атрибути, стилі, класи. Таким чином сторінка, яка поступово завантажується, може змінюватися під час взаємодії із зображеннями, текстом, формами, меню тощо, без потреби оновлювати весь документ. Це створює враження живої, адаптивної вебсторінки.

JavaScript блискуче підтримує обробку подій — кожна дія користувача є сигналом, на який мова може відреагувати. Це створює інтерактивні інтерфейси, що змінюються у відповідь на дії. Складні механізми реального часу, анімації, адаптація до різних розміщень компонентів, інтерактивний контент — все це про JavaScript.

Асинхронне програмування відіграє окрему роль. Завдяки об'єктам Promise, конструкції async/await і колбекам, JavaScript може обробляти операції, які займають тривалий час: запити до серверів, файлів, БД, апаратних сенсорів, без блокування основного потоку виконання. У браузері важливо уникнути "зависання" інтерфейсу через тривалі дії, тому асинхронні механізми дозволяють інтерфейсу залишатися резонансним і чуйним.

Ще одна важлива роль JavaScript — взаємодія з HTTP API. Програми можуть надсилати запити до серверів для отримання ресурсів, відправлення даних, синхронізації. Реактивний підхід дозволяє візуалізувати дані у реальному часі, оновлювати інтерфейс під час минливих подій, не потребуючи повного перезавантаження сторінки. На JS будуються односторінкові програми, де події користувача, навігація і дані є частиною єдиного потоку взаємодії.

Окремий пласт екосистеми JavaScript — модульність і організація коду. Через CommonJS, AMD, ES modules можна поділяти функціональність на окремі файли і репозиторії. Це дозволяє ізолювати логіку, створювати повторно використовувані модулі, реалізувати dependency injection, оптимізувати збірку. Інструменти типу Webpack чи Rollup поєднують код, мінімізують, додають sourcemap, спрощують розгортання.

У розвиток JavaScript активно включилися фреймворки, які забезпечили додаткову інфраструктуру для створення масштабованих застосунків. React, Angular, Vue.js надають архітектуру, компоненти, систему налаштування маршрутизації, управління станом, шаблонізацію, двосторонню прив'язку даних. Але вони базуються на ядрі JavaScript, і без неї не існувало б ні однієї з бібліотек.

JavaScript також стало середовищем програмування для сервера, що забезпечило тотожність мови на фронтенді і бекенді. Серверні програми

обробляють запити, виконують операції з БД, реалізують внутрішню логіку, а для клієнта використовують той самий синтаксис, структури, середовище розробки. Цей підхід прискорює розробку, оскільки не потребує вивчення та підтримки різних стеків.

Крім того, мова має значну роль у розбудові інструментів для тестування, збірки, автоматизації. Сценарії миграції, миг аналізу логів, запуск тестів, створення CI/CD pipeline — все це пишеться на JavaScript, і розробник може покривати весь цикл розробки мовою, яку він вже знає.

Що ще важливо, JavaScript підтримує низку API браузера: робота з localStorage або IndexedDB, доступ до геолокації, захоплення файлів із пристрою, push-сповіщення, вебанімації, підтримка офлайн режиму. Це дозволяє створювати прогресивні вебдодатки — PWA, які поводяться як нативні програми.

Використання JavaScript дає розробнику єдину консоль відладки, інструменти розробника в браузері, налагодження step-by-step, профайлінг продуктивності. Це спрощує життєвий цикл коду: від написання до виявлення помилок.

У підсумку JavaScript — це універсальна, сучасна мова програмування з широтною екосистемою, яка дозволяє створювати інтерфейси, які відчуваються природними, живими та інтерактивними, керувати даними, підтримувати складну бізнес-логіку, об’єднати клієнтську і серверну частини у єдину платформу. Її синтаксичні можливості, асинхронна модель, підтримка подій, динамічна самеоб'єктна модель, модулі, інтеграція з API, тестування, інструментарій середовища роблять її необхідною основою сучасних вебзастосунків. Без JavaScript не було б сучасного інтернету — такого, яким ми його знаємо.

JavaScript програмних засобах

JavaScript є основою сучасних вебдодатків. Від **платформ електронної комерції** до **вебсайтів соціальних мереж** він відіграє вирішальну роль у покращенні користувацького досвіду.

- **Вебсайти електронної комерції**: Інтернет-магазини, такі як Amazon та eBay, використовують JavaScript для динамічної фільтрації продуктів, оновлення кошика для покупок та інтерактивного оформлення замовлення. Такі функції, як "рекомендовані товари" та "оновлення цін в реальному часі", працюють на основі логіки, створеної за допомогою JavaScript.
- **Платформи соціальних мереж**: Такі вебсайти, як Facebook, Twitter та Instagram, використовують JavaScript для надання сповіщень у режимі реального часу, динамічних стрічок та швидкої взаємодії (наприклад, вподобання публікації або додавання коментаря без оновлення сторінки).
- **Вебдодатки**: Інструменти для підвищення продуктивності, такі як Google Docs, Notion і Trello, значною мірою покладаються на JavaScript для спільної роботи в режимі реального часу, автоматичного збереження та перетягування.

• **Розважальні та потокові сервіси**: Такі платформи, як YouTube, Netflix і Spotify, використовують JavaScript для керування відтворенням відео/аудіо, динамічного завантаження контенту та персоналізованих рекомендацій.

• **Інтерактивні форми та дашборди**: Банківські портали, медичні інформаційні панелі та системи підтримки клієнтів використовують JavaScript для перевірки форм, візуалізації даних і створення динамічного контенту.

Без JavaScript цим платформам не вистачало б інтерактивної та динамічної поведінки, якої користувачі очікують у сучасному цифровому світі.

Керування елементами вебсторінок

Об'єктна модель документа (DOM) - це програмний інтерфейс, який представляє HTML-документ у вигляді структурованого дерева елементів. JavaScript може взаємодіяти з DOM для динамічного оновлення вмісту сторінки, реагування на дії користувача та зміни стилів елементів у режимі реального часу.

Кожен елемент HTML є частиною DOM і може бути змінений або видалений за допомогою JavaScript. Наприклад, JavaScript може:

• Динамічно змінити вміст абзацу.

• Змінити стилі CSS елемента, щоб змінити його вигляд.

• Приховувати або показувати елементи на основі дій користувача.

• Створювати нові HTML-елементи та динамічно додавайти їх на сторінку.

Маніпуляції з DOM мають важливе значення для створення інтерактивних вебдодатків. Замість того, щоб перезавантажувати всю вебсторінку, JavaScript забезпечує цільове оновлення окремих елементів, підвищуючи ефективність і швидкість взаємодії з вебсторінкою.

Приклади використання керування DOM

1. **Перевірка форм** - запобігання надсиланню користувачами неповних або некоректних даних форм.

2. **Динамічне оновлення контенту** - автоматичне оновлення частин сторінки (наприклад, стрічки новин, ціни на акції).

3. **Інтерактивні навігаційні меню** - розгортання/згортання пунктів меню при наведенні курсору або кліку.

4. **Анімація та переходи** - зникаючі елементи, динамічне переміщення елементів на екрані.

5. **Взаємодія на основі подій** - запуск дій, коли користувачі натискають кнопки, прокручують або натискають клавіші.

Маніпуляції з DOM є основою інтерфейсного JavaScript-програмування, що дозволяє розробникам створювати динамічний інтерфейс без додаткових запитів до сервера.

Наступний приклад змінює вміст елемента <p> при натисканні кнопки:

```
<p id="greet">Привіт, гість!</p>
<button onclick="changeText()">Змінити текст</button>

<script>
  function changeText() {
    document.getElementById("greet").textContent = "Ласкаво просимо!"
  }
</script>
```

Рис.2.1. Приклад зміни тексту елемента

Також, JavaScript може створювати нові HTML-елементи або видаляти їх зі сторінки динамічно.

```
let newItem = document.createElement("li");
newItem.textContent = "Новий пункт списку";
document.getElementById("myList").appendChild(newItem);
```

Рис.2.2. Приклад створення нового елементу сторінки

Обробка подій

Обробка подій - це основна концепція JavaScript, яка дозволяє вебпрограмам реагувати на дії користувача. **Події** відбуваються щоразу, коли користувач взаємодіє з вебсторінкою - наприклад, натискає кнопку, наводить курсор на елемент або вводить дані в поле введення. JavaScript може виявити ці події і виконати певний код у відповідь.

Наприклад, натискання кнопки **"Відправити"** на формі викликає подію, яку JavaScript може перехопити, щоб підтвердити правильне введення даних користувачем перед відправленням. Також, прокрутка сторінки вниз може викликати подію, яка динамічно завантажує більше даних для відображення, як це відбувається на вебсайтах з нескінченною прокруткою.

JavaScript має способів створення слухачів подій, що стосуються окремих елементів, гарантуючи, що певний код виконується, коли відбувається певна подія. Метод addEventListener є найпоширенішим підходом, що дозволяє розробникам відокремити структуру HTML від логіки JavaScript.

Основні події JavaScript:

- **Події натискання (onclick)** - спрацьовують, коли користувач натискає на елемент.
- **Події миші (наведення, відведення миші)** - спрацьовують, коли користувач наводить курсор миші на елемент або віддаляється від нього.
- **Події клавіатури (натискання клавіші вниз, натискання клавіші вгору)** - спрацьовують, коли користувач вводить дані у полі введення.
- **Події форми (onsubmit, onchange)** - використовуються для валідації та обробки відправлених форм.
- **Події прокрутки (onscroll)** - виявлення, коли користувач прокручує сторінку, щоб запустити динамічне завантаження контенту.

Обробка подій має вирішальне значення для того, щоб вебдодатки були інтерактивними та швидко реагували. Без неї користувачам довелося б вручну оновлювати сторінки або переходити через кілька екранів, щоб побачити оновлення.

Приклад коду для відстеження кліку наведений нижче:

```
document.getElementById("myBtn").addEventListener("click", function() {
  alert("Ви натиснули кнопку!");
});
```

Рис.2.3. Приклад коду для відстеження кліку

Цикли та умовні оператори JavaScript

Вебдодатки часто потребують логіки для визначення способу відображення або обробки контенту. JavaScript реалізує **умовні оператори** (if-else) для виконання різних дій на основі певних умов.

Наприклад, інтернет-магазин може використовувати оператор if для перевірки наявності товару на складі, перш ніж дозволити користувачеві додати його до кошика. Якщо товару немає в наявності, на екрані з'явиться відповідне повідомлення.

```
let stock = 10;
if (stock > 0) {
    console.log("Product is available!");
} else {
    console.log("Out of stock.");
}
```

Рис.2.4. Приклад оператора if-else

Цикли (for, while) дозволяють JavaScript ефективно виконувати повторювані завдання. Замість того, щоб вручну писати код для кожного елемента в списку, цикл може ітераційно переглядати колекцію елементів, динамічно оновлюючи або змінюючи їх.

```
let items = ["Apple", "Banana", "Cherry"];
for (let i = 0; i < items.length; i++) {
    console.log("Item: " + items[i]);
}
```

Рис.2.5. Приклад циклу для перебору елементів списку

Наведені програмні конструкції дозволяють розробникам створювати більш динамічні та інтелектуальні додатки, зменшуючи об’єм написаного коду і підвищуючи продуктивність його роботи.

Чому JavaScript необхідний для сучасної веброзробки

JavaScript - це необхідний інструмент для створення сучасних інтерактивних вебдодатків. Він дозволяє:

- **Динамічне оновлення контенту** - підвищує адаптивність вебсайтів без перезавантаження сторінки.
- **Взаємодія з користувачами** - покращення взаємодії за допомогою форм, кнопок та анімації.
- **Покращений користувацький досвід** - забезпечення плавної, інтуїтивно зрозумілої навігації.
- **Функції реального часу** - увімкнення чату, сповіщень та оновлень у реальному часі.
- **Повностекова розробка** - розробка клієнтської частини (за допомогою таких фреймворків, як React, Vue, Angular) та серверної частини (за допомогою Node.js).

Без JavaScript вебсайти були б статичними, вимагали б постійних ручних дій користувачів для оновлення контенту. Можливість iavaScript маніпулювати DOM, обробляти події та виконувати логіку робить його критично важливим компонентом сучасної веброзробки.

Порядок виконання роботи

Крок 1: Налаштування JavaScript

Перш ніж реалізувати керування елементами з DOM, циклами та умовами, вам потрібно включити файл JavaScript в HTML-документ. Це гарантує, що браузер завантажить і виконає код JavaScript під час відображення сторінки.

1. Зв'язування **файлу JavaScript з HTML.** Щоб включити JavaScript код, потрібно додати тег <script> безпосередньо перед закриваючим тегом </body> у HTML-файлі, зв'язавши його з вашим зовнішнім файлом JavaScript.

```
<body>
    <script src="script.js"></script>
</body>
```

Рис.2.6. Зв'язування файлу JavaScript з HTML

Це гарантує, що JavaScript завантажується після вмісту HTML, тому всі елементи доступні для керування елементами.

Крок 2: Керування елементами DOM

Керування елементами DOM дозволяють взаємодіяти з HTML-документом і змінювати його за допомогою JavaScript.

1. **Виділення елементів HTML.** Щоб взаємодіяти з елементами HTML, спочатку потрібно виділити їх за допомогою JavaScript. Використовуйте методи

document.getElementById(), document.querySelector(), а також методи document.querySelectorAll() для вибору окремих елементів або груп елементів.

```
// Select an element by ID
const header = document.getElementById('header');

// Select an element by class
const navLinks = document.querySelectorAll('.nav-link');
```

Рис.2.7. Вибір окремих елементів або груп елементів

2. **Зміна вмісту.** Вибравши елемент, можна змінити його вміст за допомогою властивості .innerHTML або .textContent для звичайного тексту.

```
// Changing the header text
header.innerHTML = 'Welcome to My Updated Website!';

// Changing text of all navigation links
navLinks.forEach(link => {
  link.textContent = 'New Link Text';
});
```

Рис.2.8. Зміна вмісту елемента

3. **Модифікація атрибутів** елементів. Також є можливість змінювати атрибути елементів за допомогою .setAttribute() для таких атрибутів, як href, src тощо.

```
// Changing the 'href' attribute of a link
const link = document.querySelector('a');
link.setAttribute('href', 'https://newurl.com');
```

Рис.2.9. Модифікація атрибутів елементів

Крок 3: Обробка подій

JavaScript дозволяє взаємодіяти з користувачами за допомогою подій, таких як кліки, натискання клавіш, заповнення форм тощо. Ви можете додавати слухачі подій до елементів і визначати функції, які будуть виконуватися, коли ці події відбуваються.

1. **Додавання слухачів подій.** Потрібно використати addEventListener() для додавання обробників подій до елементів. Можна вказати тип події (наприклад, клік, відправлення, наведення миші) і функцію зворотного виклику, яка буде виконуватися при її спрацьовуванні.

```
// Example: Adding a click event to a button
const button = document.getElementById('myButton');
button.addEventListener('click', function() {
  alert('Button clicked!');
});
```

Рис.2.10. Додавання слухачів подій

2. **Обробка форм.** Якщо вебсторінка містить форми, є можливість прослуховувати подію відправлення і запобігти відправленню форми за замовчуванням, щоб обробити її за допомогою JavaScript.

```
const form = document.getElementById('myForm');
form.addEventListener('submit', function(event) {
  event.preventDefault(); // Prevent form from submitting
  alert('Form submitted!');
});
```

Рис.2.11. Обробка форм

Крок 4: Цикли в JavaScript

Цикли дозволяють повторювати дії певну кількість разів або до виконання певної умови. У JavaScript найпоширенішими циклами є for, while та forEach.

1. **Використання циклу** for. Цикл for ідеально підходить, коли відомо, скільки разів потрібно повторити дію.

```
// Loop through an array of numbers and log each number
const numbers = [1, 2, 3, 4, 5];
for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}
```

Рис.2.12. Використання циклу for

2. **Використання циклу** while. Цикл while використовується, коли потрібно повторити дію, доки умова є істинною.

```
// Example: Log numbers until we reach 5
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

Рис.2.13. Використання циклу while

3. **Використання forEach.** Цикл forEach використовується для перебору масивів. Він більш лаконічний, ніж звичайний цикл for, і особливо корисний при роботі з масивами.

```
// Loop through an array and log each element
const fruits = ['apple', 'banana', 'cherry'];
fruits.forEach(function(fruit) {
  console.log(fruit);
});
```

Рис.2.14. Використання forEach

Крок 5: Умовні оператори

Умовні оператори дозволяють керувати потоком вашого коду на основі певних умов. У JavaScript основними умовними операторами є if, else if та else.

1. **Базовий оператор if**. Оператор if перевіряє умову і виконує код, якщо умова істинна.

```
const age = 18;
if (age >= 18) {
  console.log('You are an adult.');
}
```

Рис.2.15. Базовий оператор if

2. **else if та else.** Існує можливість розширити оператор if за допомогою else if для перевірки декількох умов, а else для виконання коду, коли жодна з умов не є істинною.

```
const temperature = 30;
if (temperature > 30) {
  console.log('It\'s very hot outside!');
} else if (temperature > 20) {
  console.log('The weather is warm.');
} else {
  console.log('It\'s a bit chilly.');
}
```

Рис.2.16. Використання else if та else

3. **Тернарний оператор.** Більш стислим способом запису умов є тернарний оператор, який можна використовувати замість простих операторів if-else.

```
const age = 18;
const status = (age >= 18) ? 'Adult' : 'Minor';
console.log(status);
```

Рис.2.17. Використання тернарного оператора

Крок 6: Поєднання концепцій

Тепер, після розглянутих способів керування DOM, обробки подій та використання циклів й умов, настав час об'єднати ці навички в інтерактивній вебсторінці.

Приклад: Зміна кольору фону при натисканні кнопки. У цьому завданні ви додасте слухача подій до кнопки, яка змінює колір фону сторінки при натисканні. Можна використовувати цикл і умовний оператор для циклічного перегляду списку кольорів.

```
<button id="colorButton">Change Color</button>

<script>
  const colors = ['red', 'blue', 'green', 'yellow'];
  let currentColor = 0;

  document.getElementById('colorButton').addEventListener('click', function() {
    document.body.style.backgroundColor = colors[currentColor];
    currentColor = (currentColor + 1) % colors.length;
  });
</script>
```

Рис.2.18. Зміна кольору фону при натисканні кнопки

Цей приклад демонструє:

- Маніпуляції з DOM (зміна кольору фону).
- Обробка подій (натискання кнопки).
- Цикли (циклічне чергування кольорів).

Лабораторне завдання

Завдання 1: Керування DOM за допомогою циклів та умовної логіки

Опис: Вибір елементів та їх модифікації за допомогою циклів та умовних операторів для створення динамічної поведінки.

Кроки:

1. Використати JavaScript для виділення декількох елементів (наприклад, всіх елементів у списку або всіх зображень).
2. Використати цикл for для циклічного перебору колекції елементів (наприклад, для зміни кольору фону кожного елемента списку при завантаженні сторінки).
3. Застосувати оператори if-else для зміни вмісту або стилю елементів на основі певних умов (наприклад, зміна текстового вмісту елемента на основі його індексу або атрибутів даних).
4. Використати **querySelectorAll**, щоб вибрати кілька елементів і перебирати їх, динамічно змінюючи їхній стиль або вміст.

Завдання 2: Обробка подій та динамічні оновлення

Опис: Додавання інтерактивності з подіями та оновлення DOM на основі дій користувача.

Кроки:

1. Створити кнопку, яка при натисканні перемикає видимість розділу сторінки (наприклад, показує/приховує додатковий опис, розділ коментарів або меню).
2. Реалізувати **логіку if-else** для перевірки стану видимості елемента (наприклад, якщо секція прихована, показати її; якщо вона видима, сховати її).
3. Використати **цикл for для** додавання обробників подій до декількох елементів (наприклад, кнопок або посилань у навігаційному меню). Кожна кнопка повинна змінювати вміст певної області на сторінці при натисканні.
4. Створити **ефект наведення**, коли при наведенні на елемент змінюється його вміст або стиль, використовуючи обробник подій і умовну логіку (наприклад, наведення на пункт меню, щоб показати його опис).

Завдання 3: Динамічне керування контентом

Опис: Додавання динамічної функціональності до вебсторінки, наприклад, створення або оновлення вмісту на основі введення користувачем даних.

Кроки:

1. Створити форму з полями для введення (наприклад, ім'я, електронна пошта або коментар) і кнопкою надсилання.
2. Використати **JavaScript для збору даних з форми** та **оператор if-else** для перевірки даних (наприклад, щоб переконатися, що користувач не заповнив порожні поля).
3. Після валідації використати **маніпуляції з DOM** для динамічного додавання введених користувачем даних до розділу сторінки (наприклад, відобразити коментар під статтею або дані користувача в області підтвердження).
4. Використати **цикл for для** динамічного відображення списку елементів (наприклад, дописів блогу або пунктів меню), з кожним з яких можна взаємодіяти (наприклад, при кліку на заголовку допису, відображеться його повний зміст).

Контрольні запитання

1. Яку роль відіграє JavaScript у веброзробці та як він взаємодіє з HTML і CSS?
2. Що таке об'єктна модель документа (DOM) і як JavaScript може змінювати її? Наведіть приклад.
3. Як працює обробка подій у JavaScript? Наведіть приклад використання addEventListener().
4. Які є основні типи циклів у JavaScript і коли варто використовувати for, while та forEach?
5. Як умовні оператори (if-else) допомагають створювати динамічну поведінку вебсторінки? Наведіть приклад.

6. Коли і з якою метою була створена мова програмування JavaScript?
7. Чим відрізняється JavaScript від HTML і CSS у контексті веброзробки?
8. Які особливості має JavaScript як мова високого рівня?
9. Які переваги асинхронного програмування в JavaScript?
10. Що таке DOM і як JavaScript взаємодіє з ним?
11. Як за допомогою JavaScript змінити текст усередині HTML-елемента з ідентифікатором message?
12. Які методи DOM дозволяють створювати або видаляти елементи сторінки?
13. Як змінити стиль елемента сторінки через JavaScript?
14. Чим корисні маніпуляції з DOM для вебдодатків?
15. Що таке подія у JavaScript?
16. Як працює метод addEventListener()? Які переваги його використання?
17. Які типи подій існують у JavaScript?
18. Яка різниця між подіями onclick і onmouseover?
19. Як організувати перевірку введених даних у форму перед її надсиланням?
20. Які типи циклів реалізовано в JavaScript?
21. Коли варто використовувати цикл for, а коли while?
22. Яка структура циклу for? Що означає кожна частина?
23. Як за допомогою циклу перебрати масив чисел і вивести тільки парні?
24. Які переваги використання циклів у роботі з елементами сторінки?
25. Яка загальна структура умовного оператора if-else?
26. Як JavaScript обробляє вкладені умови (if всередині if)?
27. Яка різниця між == і === у умовах?
28. Як можна використати умовний оператор для перевірки заповненості поля введення?
29. Який результат отримає користувач, якщо умова не виконується?
30. Що таке Promise і яку проблему він вирішує?
31. Для чого використовується async/await у JavaScript?
32. Як можна реалізувати динамічне завантаження контенту при прокрутці сторінки?
33. У чому перевага асинхронної обробки подій для користувацького інтерфейсу?
34. Як JavaScript дозволяє реалізувати функціональність реального часу, наприклад, оновлення новинної стрічки?

Вимоги до оформлення звіту

- Титульна сторінка
- Тема, мета лабораторної роботи
- Результати виконання лабораторної роботи
    - Зображення частини розробленої вебсторінки, що відображає елементи інтерфейсу користувача використовуючи цикл відповідно до індивідувального завдання.
    - Зображення нової кнопки вебзастосунку доданої відповідно до індивідувального завдання.
    - Зображення результату виконання функції при натисненні на додану кнопку.
    - Зображення інтерактивної частини вебзастосунку до взаємодії з користувачем.
    - Зображення інтерактивної частини вебзастосунку після взаємодії з користувачем та виконання нової розробленої інтерактивної функції вебзастосунку.

Варіанти завдань

Варіант 1

Вебсайт ресторану

1. Використати цикл for для відображення пунктів меню у таблиці
2. До кожного пункту меню додати кнопку «Інгредієнти», а також прихований блок з списком інгредієнтів страви та змінювати видимість даного блоку при натисканні кнопки «Інгредієнти».
3. На сторінці Про нас реалізувати форму коментарів, де користувачі можуть залишати відгуки про ресторан та динамічно відображати їх після додавання окремим списком.

Варіант 2

Вебсайт організації онлайн-ігор

1. Використати цикл while для генерації випадкового списку рекомендованих ігор, що змінюються при оновленні сторінки.
2. Додати кнопку "Додати до улюблених", яка при натисканні змінює розмір картки гри та додає рамку іншого кольору. Використати подію подвійного кліку (dblclick) для взаємодії.
3. На сторінці профілю реалізувати відображення анімованого прогресу користувача, що показує, скільки ще потрібно ігор для отримання наступної нагороди.

Варіант 3

Вебсайт платформи для навчальних курсів

1. Використати do...while для автоматичного оновлення розкладу занять у певний період часу.

2. Додати кнопку "Розпочати курс", яка при натисканні змінює положення блоку з описом курсу на сторінці (переміщує вище). Додати кнопку "Курс пройдено", яка при натисканні змінює колір фону блоку курсу та відображає текст «Пройдено».

3. Реалізувати візуальну шкалу прогресу навчання, яка заповнюється залежно від кількості пройдених курсів.

Варіант 4

Вебсайт платформи для бронювання турів

1. Використати while для відображення турів у списку.

2. Додати кнопку "Забронювати тур", яка при натисканні збільшує розмір картки туру (робить її більш виразною) та додає в "Мої бронювання".

3. Реалізувати інтерактивну мапу популярних туристичних місць, що дозволяє кліком на маркер переглядати коротку інформацію про кожне місце.

Варіант 5

Вебсайт онлайн-магазину книг

1. Використати цикл for для генерації списку рекомендованих книг, який змінюється при оновленні сторінки.

2. Додати кнопку "Додати до кошика", яка при натисканні додає книгу в кошик. Кнопка змінює колір після додавання товару.

3. Реалізувати функціональність для підрахунку вартості товарів у кошику в реальному часі, коли кількість товару змінюється.

Варіант 6

Вебсайт платформи для оренди автомобілів

1. Використати while для генерації списку автомобілів, які доступні для оренди на даний момент.

2. Додати кнопку "Забронювати", яка при натисканні додає автомобіль у список бронювань користувача. Кнопка змінює колір і додає ефект анімації при натисканні.

3. Реалізувати календар для вибору дати початку та кінця оренди автомобіля.

Варіант 7

Вебсайт платформи для оренди житла

1. Використати do...while для виведення списку доступних квартир.

2. Додати кнопку "Забронювати", яка при натисканні додає квартиру в список бронювань користувача, а також змінює статус доступності цієї квартири (позначає її як заброньовану).

3. Реалізувати інтерактивну мапу для відображення місця розташування квартир на карті. При натисканні на маркер відображати додаткову інформацію про квартиру.

Варіант 8

Вебсайт платформи для онлайн-бронювання квитків на заходи

1. Використати while для відображення лише майбутніх подій.

2. Додати кнопку "Забронювати квиток", яка при натисканні додає подію в розділ "Мої бронювання" і змінює статус кнопки на "Забронювано".

3. Реалізувати вибір кількості квитків при бронюванні та підрахунок загальної вартості квитків.

Варіант 9

Вебсайт платформи для організації волонтерських ініціатив

1. Використати while для відображення лише актуальних ініціатив, які ще не завершилися.

2. Додати кнопку "Приєднатися", яка при натисканні додає ініціативу в розділ "Мої ініціативи" і змінює статус кнопки на "Ви приєдналися".

3. Реалізувати оновлення значення кількості волонтерів, яких ще потрібно залучити, після приєднання користувача.

Варіант 10

Вебсайт онлайн-магазину спортивних товарів

1. Використати while для відображення тільки доступних до покупки товарів.

2. Додати кнопку "Додати до кошика", яка при натисканні додає товар до кошика та змінює статус кнопки на "Товар у кошику".

3. Реалізувати можливість зміни кількості товарів у кошику, а також автоматично оновлювати загальну суму покупки.

Варіант 11

Вебсайт платформи для пошуку робочих вакансій

1. Використати цикл for для генерації списку вакансій на основі даних із бази.

2. Додати кнопку "Подати заявку", яка при натисканні змінює статус вакансії на "Подано", відображаючи повідомлення про успішну подачу заявки.

3. Реалізувати систему фільтрації вакансій за категорією, містом і рівнем зарплати, застосовуючи випадаючі списки.

Варіант 12

Вебсайт платформи для замовлення їжі

1. Використати цикл for для генерації списку страв у меню та кошику, цикл while у списку "Мої замовлення".

2. Додати кнопку "Додати в кошик", яка змінює колір після натискання та додає страву в кошик.

3. Реалізувати таймер очікування доставки замовлення після оформлення.

Варіант 13

Вебсайт блогу для подорожей

1. Використати цикл while для виводу статей у списку.

2. Додати кнопку "Подобається", яка змінює стиль статті після натискання.

3. Реалізувати можливість коментування статей та динамічне оновлення списку коментарів.

Варіант 14

Вебсайт інтерактивної платформи для вивчення мов

1. Використати масив об'єктів та цикл for для генерації списку уроків.
2. Додати кнопку "Відзначити як пройдений", яка змінює стиль уроку після натискання та зберігає стан у локальному сховищі.
3. Реалізувати таймер щоденного навчання, який відстежує витрачений час.

Варіант 15

Вебсайт платформи для вивчення історії через інтерактивні події

1. Використати масив історичних подій та цикл while для генерації контенту на сторінці хронології.
2. Додати кнопку "Дізнатися більше" на часовій шкалі, яка розкриває додаткові деталі події у модальному вікні.
3. Реалізувати тест для перевірки знань, що відкривається натисканням на кнопку «Тест» у розділі «Події».

Варіант 16

Вебсайт симулятора управління власним стартапом

1. Використати цикл map для відображення списків конкурентів та інвесторів.
2. Додати кнопку форму "Моделювання бізнес процесу", яка змінює параметри стартапу (наприклад, збільшує прибуток чи кількість працівників).
3. Реалізувати динамічний звіт у вигляді графіку та таблиць порівняння реальних та змодельованих параметрів бізнесу (доходи, витрати, кількість працівників, тощо).

Варіант 17

Вебсайт платформи для створення та управління персональними цілями

1. Використати методи масивів (map, filter) для управління списком цілей.
2. Додати кнопку "Виконано", яка змінює статус цілі та оновлює прогрес.
3. Реалізувати щоденний мотивуючий таймер, що нагадує про заплановані завдання.

Варіант 18

Вебсайт платформи для віртуальних хакатонів та змагань із програмування

1. Використати цикл forEach для генерації списку учасників у рейтингу.
2. Додати кнопку "Подати заявку", яка змінює стиль після натискання та додає користувача до списку учасників.
3. Реалізувати таймер до завершення хакатону, що оновлюється в реальному часі.

Варіант 19

Вебсайт симулятора управління будівництвом містом

1. Використати цикл do..while для виведення інформації про будівлі та ресурси.
2. Додати кнопку "Покращити будівлю", яка змінює її вигляд (фото) та покращує характеристики, але забирає ресурси у міста.
3. Реалізувати динамічне зменшення ресурсів міста після створення нових чи покращення старих будівель.

Варіант 20

Вебсайт симулятора космічної експедиції

1. Використати цикл for для генерації списку доступних експедицій.
2. Додати кнопку "Почати місію", яка змінює статус експедиції та додає запис у журнал подорожей.
3. Реалізувати таймер тривалості польоту, що оновлюється в реальному часі.

Варіант 21

Вебсайт платформи для моніторингу здоров'я та фізичної активності

1. Використати цикл for виведення інформації про тренування.
2. Додати кнопку "Почати тренування", яка змінює статус тренування та оновлює прогрес.
3. Реалізувати журнал тренувань, що показує список проведених тренувань, тип проведеного тренування та тренувальний час.

Варіант 22

Вебсайт платформи для онлайн-курсу з фотографії

1. Використати цикл while для виведення інформації про уроки та фотографії.
2. Додати кнопку "Завантажити фотографію", яка додає нову фотографію до галереї.
3. Реалізувати фільтрацію фотографій за типами знімків (портрети, пейзажі, макрозйомка тощо).

Варіант 23

Вебсайт платформи для оренди спортивного обладнання

1. Використати цикл do..while для виведення даних про обладнання та оренди.
2. Додати кнопку "Орендувати", яка змінює колір після натискання та додає товар до кошика.
3. Реалізувати функціонал для додавання дати початку та закінчення оренди.

Варіант 24

Вебсайт платформи для планування подорожей

1. Використати цикл do..while для виведення інформації про місця та подорожі.
2. Додати кнопку "Додати до моїх подорожей", яка додає місце до запланованої подорожі.
3. Реалізувати функціонал для розрахунку загального бюджету подорожі в залежності від вибраних місць і тривалості.

Варіант 25

Вебсайт платформи для створення та обміну рецептами

1. Використати цикли for для виведення рецептів та відгуків.
2. Додати кнопку "Додати рецепт", яка відкриває форму для створення нового рецепту.
3. Реалізувати функціонал для редагування рецептів після їх створення.